Full-Stack Take-Home — DEALPORT Admin Dashboard + Products API

Full-Stack Developer assessment · NestJS-heavy · Next.js admin UI · Timebox: **48–72 hours**

> Implement the scoped DEALPORT screens from the public Figma Community file, with a real NestJS + Prisma API for products. Do **not** build the entire kit — depth on the scoped slice beats breadth.

1. Role

Full-Stack Developer (NestJS-heavy, Next.js for admin UI). Complete within **48–72 hours**.

2. Stack (required)

- **Backend:** NestJS, TypeScript, Prisma, PostgreSQL, JWT
- **Frontend:** Next.js (App Router), TypeScript, Tailwind CSS
- **Deploy:** Public URL for frontend + backend (or monorepo deploy)

3. Design source (public Figma Community only)

Duplicate this community file to your drafts and implement **ONLY** the scoped screens:
https://www.figma.com/design/pb00zvXQ4XQo6zIh1hxko7/E-commerce-Dashboard--Admin--UI-Kits--Community-?node-id=5-1346
Brand in the kit: **DEALPORT** (emerald/green admin shell). Reference screenshots of the required UI are attached at the end of this PDF.

4. Scoped screens (in scope)

1. **Dashboard** — app shell (sidebar + top bar) plus:
   - Stat cards (sales / orders / pending)
   - "Report for this week" summary + chart area (chart library or static SVG OK)
   - Transaction table (seeded API or static data OK if documented)
   - Best selling / Top products widgets **MUST** load from NestJS product APIs
2. **Add Product** — form with basic details, pricing, inventory, categories/tags, image upload UI
   - Publish Product + Save to draft (map to product status)
   - Image upload: real upload *or* URL/base64 stub is OK if documented in README
3. **Product List** — sidebar "Product List"; table/list **MUST** be API-integrated (list + minimal search/filter)

5. Explicitly out of scope

- Order Management full flows
- Customers, Coupon Code, Brand
- Product Media / Product Reviews (beyond what's needed for Add Product images)
- Admin role / Control Authority
- Theme polish beyond matching the green primary look
- Any other screens in the kit

6. Backend requirements (NestJS)

- Modules: `auth`, `products`, `categories` (tags if the form needs them)
- JWT login for an admin/seller persona
- Prisma + PostgreSQL models: User, Product, Category (+ optional Tag, image URLs)
- Minimum endpoints:
  - `POST /auth/login` (seed an admin user)
  - `GET /products`, `POST /products`, `GET/PATCH/DELETE /products/:id` — list supports search + pagination
  - `GET /categories` (seed Electronic, Fashion, Home, etc.)
- DTOs + class-validator; Controller → Service → Prisma (no Prisma in controllers)
- Product List, Add Product, and dashboard product widgets all use this API — **no mock-only product data**

7. Frontend requirements (Next.js)

- App Router + TypeScript + Tailwind
- Layout matching DEALPORT shell (sidebar: Dashboard / Add Products / Product List with active states)
- Typed API client + Bearer token after login
- Desktop-first (~1440 design); mobile "usable" not required
- Layout fidelity: good enough match (spacing, hierarchy, table/form structure). Pixel-perfect not required.

8. Deliverables

1. GitHub repo (public or invite-only) — monorepo or separate FE/BE
2. Deployed demo URL(s)
3. README: setup, env vars (`.env.example` only — no secrets), architecture notes, seed users/passwords
4. Seed script so reviewers can log in immediately

9. Submission

GitHub URL

Live frontend URL	
Live API URL	
Seed credentials	
Hours spent (approx)	

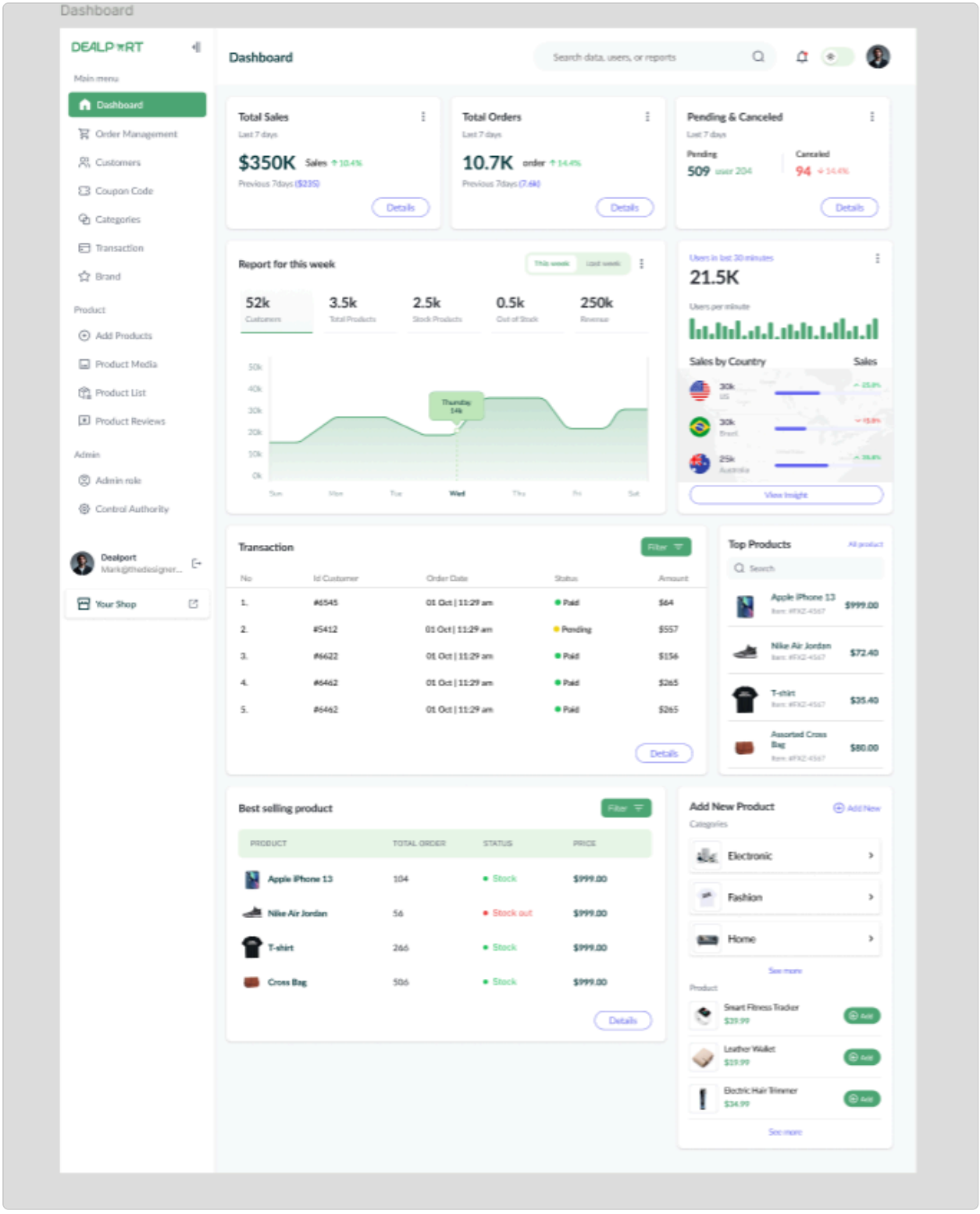
10. Notes

- Real NestJS API required for products — mock product frontends fail.
- No company-internal designs or codebases are provided; use the community Figma link above.

Reference UI screenshots follow on the next pages (Dashboard, then Add Product).

Reference — Dashboard (DEALPORT)

Implement the shell + main dashboard widgets as scoped above. Product widgets must use your NestJS products API.



Reference — Add Product (DEALPORT)

Implement this Add Product form (publish + draft). Product List must also be API-integrated.